

Università degli Studi di Milano-Bicocca



Machine Learning models for Tic-Tac-Toe game classification

Students:

Favagrossa Francesco - Mat. 933947

Rigamonti Chiara - Mat. 880780

Tosi Chiara - Mat. 933069

November 2025

Abstract

The following report presents the results of the analysis performed on the TicTacToe dataset; an analysis whose main target is to train different machine learning models to develop a robust classifier, capable of accurately classifying new instances. The analysis begins with a descriptive statistics phase, followed by the application of supervised learning methods. For classification, four models were used and compared, including: K-Nearest Neighbor (K-NN), Support Vector Machine (SVM), Gaussian Process (GP) and Random Forest (RF). During this analysis, we will show which algorithms performed best, using “accuracy” as the main metric, as it is the most suitable indicator in our context, where the classes are not so much imbalanced. We decided not to calculate indicators like “precision” and “recall”, because our context did not allow us to define whether the cost of a false positive or a false negative is higher. Finally, considerations will be made regarding which of these algorithms is best suited for our context, defining the preferred choice for deployment.

1 Descriptive analysis

1.1 Dataset structure

The analyzed dataset consists of 1215 observations, 12 numeric explanatory features and the target variable "class", also numeric. If we plot the first two features, we can see that they perfectly represent the tic-tac-toe game, but if we plot all other features, we notice that they distribute as a Uniform in (0,1), this is shown in Figure 1(a).

Since this is a binary classification dataset, the class labels are "-1" and "+1", and the class division is not perfectly homogeneous; in fact, the most frequent class is "+1," with 675 observations, while class "-1" has 540 observations. Furthermore, there are no missing data or outliers, as shown in Figure 1(b).

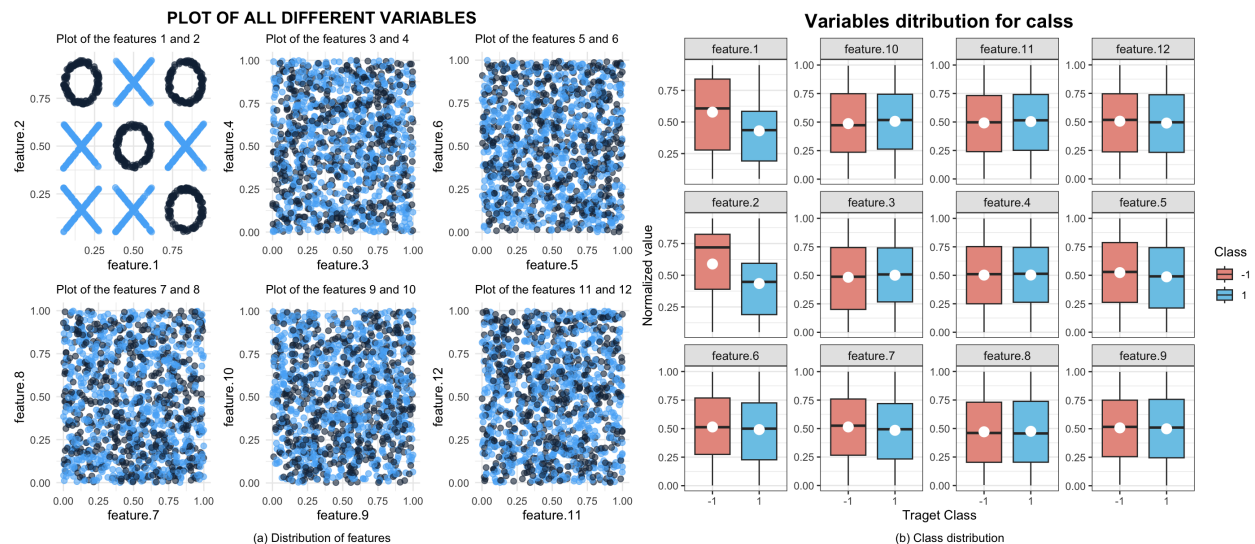


Figure 1: Dataset Analysis

1.2 Correlation Matrix

It is important to determine the presence of a possible correlation between features, since if this were to be high, it could be decided to perform a Principal Component Analysis (PCA), to reduce the dimensionality of the dataset, thus facilitating the implementation of the study. Figure 2(a) shows that none of the variables exhibit a particularly high correlation value, therefore, applying a PCA is not advisable, since, as shown in Figure 2(b), each of the identified principal components would explain approximately the same percentage of the dataset's variance.

2 Preprocessing and Data Splitting

Based on the previous reasoning, we decided to implement a single data pre-processing technique: the Min-Max Rescaling. The goal of this technique is to ensure that all features fall within a single range, ensuring their equal contribution to the model, regardless of their original scale. Furthermore, before implementing any type of machine learning analysis, it is important to perform a data splitting process, through which the original dataset is divided into two groups: the first, called the "dataset", which includes both the training set and the validation set and it is composed of 1094 observations (90%), and the second, which will be used as the "testset", that is composed of 121 observations (10%). This procedure is performed using a specific function that allows the proportion of classes to be maintained even after partitioning.

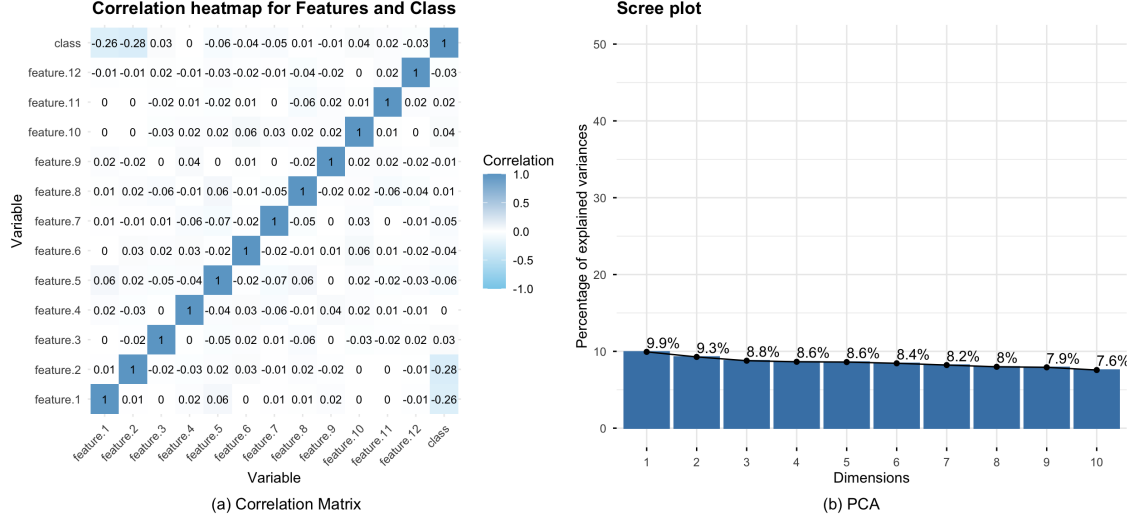


Figure 2: Correlations and variance

3 Supervised learning techniques: Crisp Models

3.1 K Nearest Neighbors (K-NN)

The analysis begins with the implementation of the K-NN algorithm, which aims to provide a classification for an unknown instance by assigning it to the class to which the majority of its closest neighbors belong. These models provide crisp predictions. It is important, before starting, to identify the “best” value of the hyperparameter k , because if it is chosen a value of k that is too low, there is a risk of overfitting. So, in order to avoid this problem, three different procedures are implemented : Hold out, K Fold Cross Validation and Bayesian Optimization. Since this is a binary classification task, an odd value for k is preferred to prevent ties, therefore only odd k values were evaluated. Figure 3 presents how different values of k , found through the first two procedures mentioned, lead to different values of accuracy. We can easily see how the level of accuracy decreases when k increases, so it was not considered necessary to test values for k higher than 51.

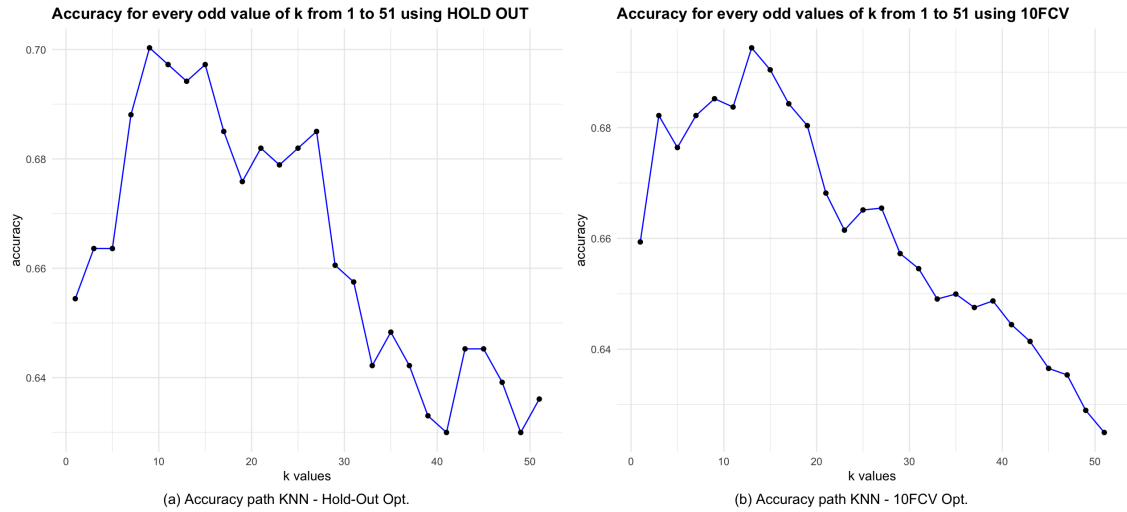


Figure 3: KNN Opt. paths

Despite all the initial considerations, we notice that the KNN algorithm does not provide sufficiently

robust results, both algorithms return a similar level of accuracy: 70% using hold-out procedure and 69.4% using 10-Fold cross validation. Furthermore, as shown in Figure 4 if we try to classify new instances contained in the set test, we notice that the algorithm is not able to correctly classify enough instances to be considered a “good classifier”.

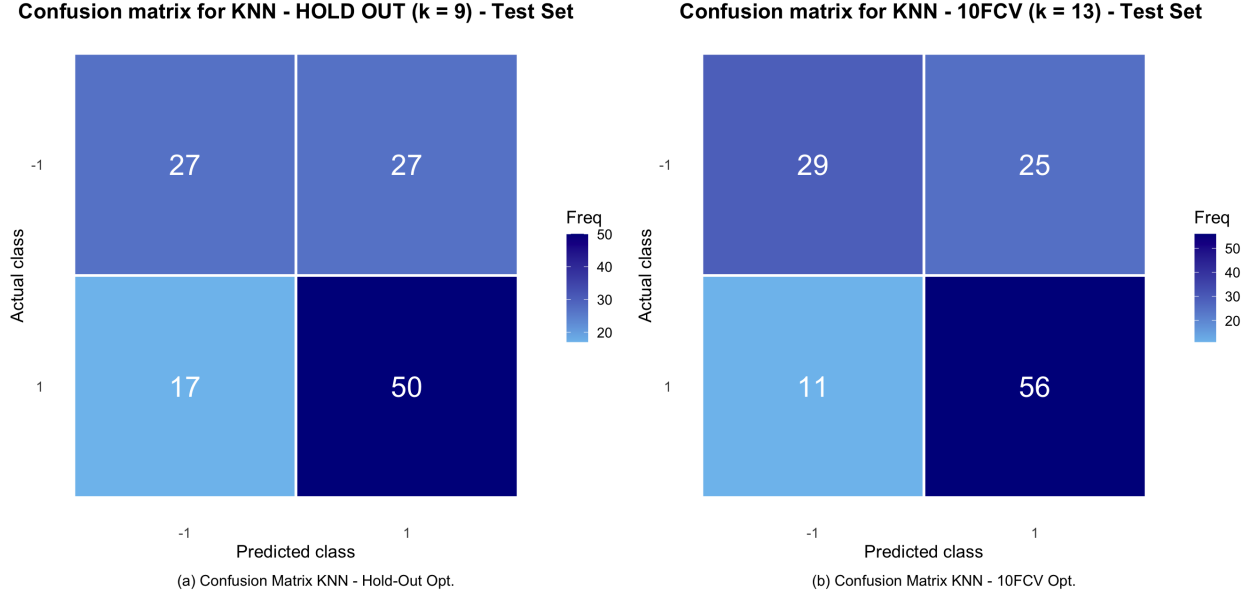


Figure 4: Confusion Matrices

We decided to shift the focus from a Grid Search to a Sequential Search, thus implementing a Bayesian Optimization. By doing so, we would have expected at least a minimum improvement in terms of accuracy, but this does not rise above 68.2%, as can be seen in Figure 5(a), where the gray lines represent the level of accuracy achieved at each iteration. The blue line in the first graph highlights the evolution of accuracy considering only its increase, instead, the blue line in the second graph provides the trend of accuracy at each value of k evaluated.

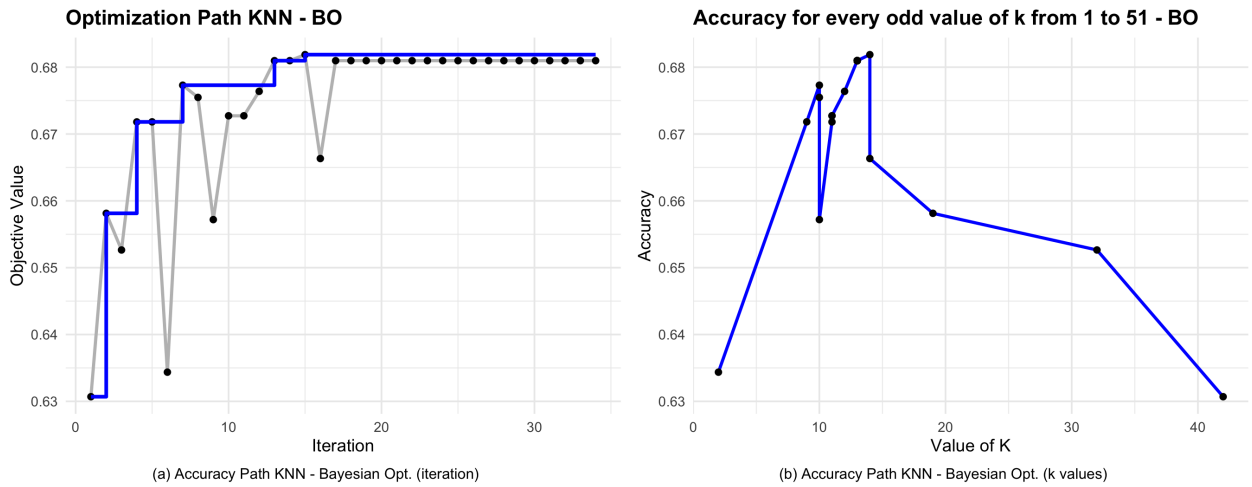


Figure 5: Optimization and accuracy

We can observe the confusion matrix, Figure 6, which clearly shows that the model does not perform well in classifying the data, demonstrating unsatisfactory results. So, we can conclude that the KNN model cannot achieve a level of accuracy that can guarantee the necessary predictive reliability, so it won't be our first choice when we want to classify unseen data. The result does not surprise us, the KNN algorithm works well if the classes are well separated in the feature space (this happens just for the first two features), and works well with low-dimensional dataset, because it does not have a real training phase, so in order to classify a new data point it must calculate the distance from all the points in the training dataset and our dataset is made up of many observations.

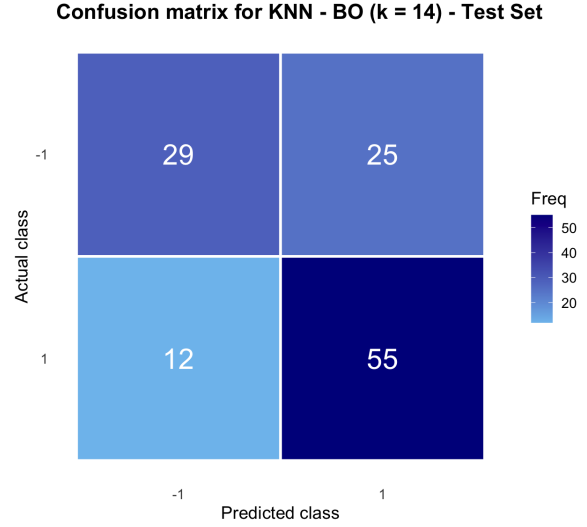


Figure 6: Confusion matrix for KNN - Best k

3.2 Support Vectors Machine (SVM)

The second algorithm presented aims to identify the hyperplane that best separates the classes, using support vectors (the observations closest to the separation region between the two classes). The SVM attempts to maximize the distance between them and the hyperplane, called the margin.

Since our data appears to be non-linear, we resort to using a kernel function, which maps the data into a higher-dimensional space, to make linear separation possible. Using Bayesian Optimization, we determined that the best kernel type to use is the “radial” one, consequently the values required by the hyperparameters c and γ were optimized, both through Bayesian Optimization and K-Fold Cross Validation, subsequently the model was trained.

The result obtained also seems not to be very promising, we were not able to find an accuracy value that exceeded 67% through Bayesian Optimization and 68.6% through K-Fold Cross Validation.

By looking at Figure 7 and Figure 8 it can be seen that the algorithm applied to the test set does not produce optimal results, the SVM might not have found the hyperplane that guarantees 0% error on the Test Set. The results obtained are justified by the fact that the algorithm uses a very high number of support vectors (813), a number that closely approaches the total number of observations in the training set. Consequently, this leads the algorithm to have a poor generalization ability on new, unseen data.

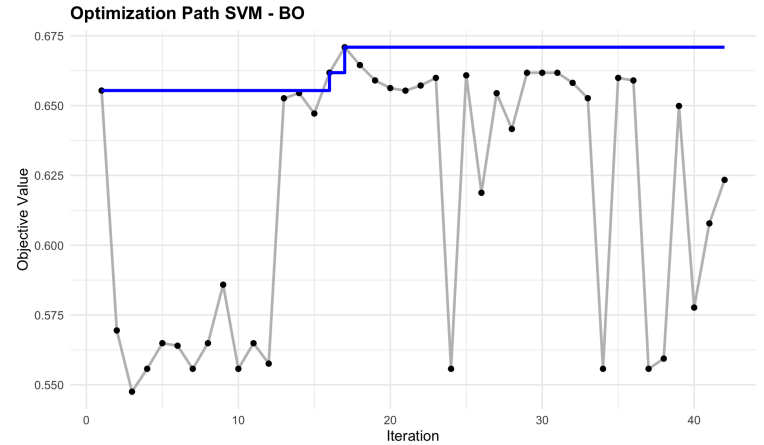


Figure 7: Opt. path SVM - BO

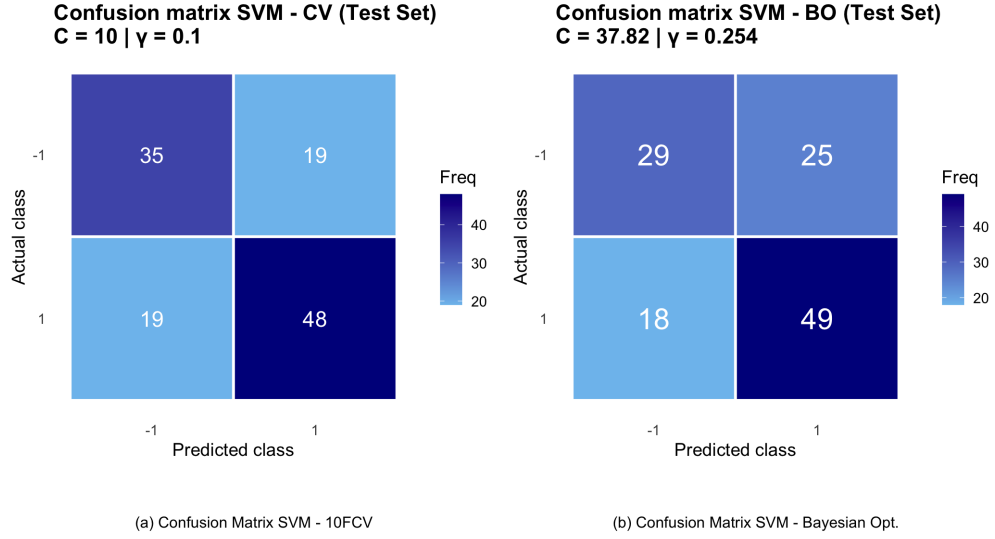


Figure 8: Confusion Matrices SVM

4 Variables selection

As mentioned in the Descriptive Analysis section, features 3 to 12 very likely follow a uniform distribution, suggesting their contribution to class prediction may not be fundamental. Their probable lack of usefulness is also evident from Figure 1(a). To perform variable selection, we employed Lasso Regression. By examining the regularization path of each variable as λ increases, we can identify which variables play a major role in prediction. While Lasso considers linear relationships, it is common practice to perform this linear selection first and then apply other techniques (linear or non-linear) to the selected variables.

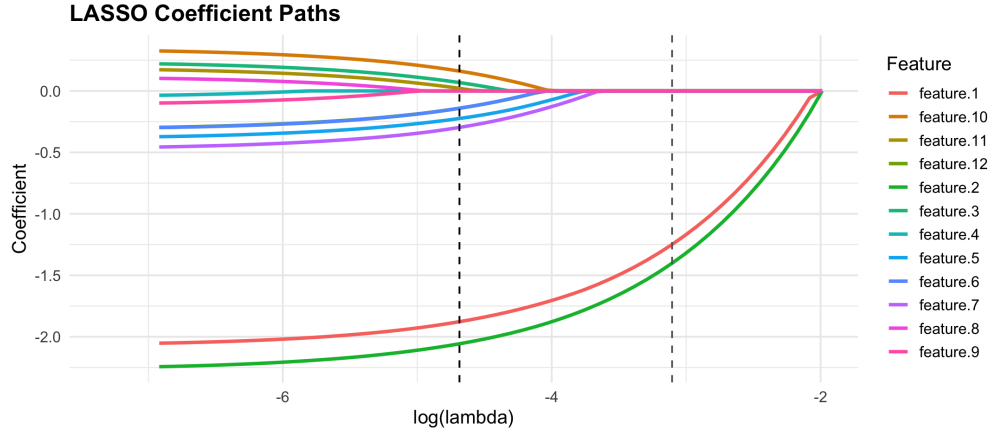


Figure 9: Variable shrinkage path for Lasso regression

Figure 9 clearly shows that features 3-12 immediately shrink to zero as $\log(\lambda)$ increases. Moreover, even for small values of $\log(\lambda)$ the coefficients of these features remain very small, with some already at zero. This behavior strongly supports their removal from the model.

Based on these results and our prior intuition about their distribution, we decided to exclude features 3-12 and re-evaluate the previously presented models using only the first two variables.

4.1 K-Nearest Neighbors on Selected Variables

Since Bayesian Optimization allows for a better exploration of parameters space, we applied only this optimization technique to the selected variables. The KNN model was retrained with the reduced feature set, and the results are highly encouraging. The dimensionality reduction had an important effect on model performance. With only two features, the neighborhood structure becomes much clearer, and the algorithm can effectively identify local patterns without the interference of noisy, uninformative variables. The optimal number of neighbors found through Bayesian Optimization allows the model to balance between overfitting and underfitting in this simplified feature space. The model achieved 100% accuracy on the test set, demonstrating that the first two features contain all the information necessary for the classification. This substantial improvement from the previous results confirms that features 3-12 were indeed adding noise rather than signal to the prediction task.

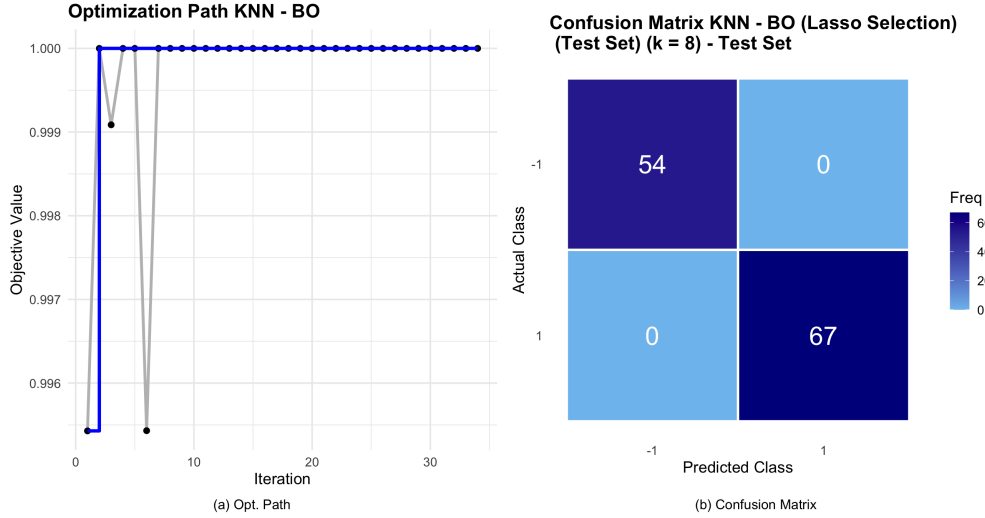


Figure 10: KNN classification results on selected variables

4.2 Support Vector Machine on Selected Variables

After reducing the feature space to only the first two variables, the SVM model was retrained and re-optimized. The radial basis function (RBF) kernel proved to be particularly efficient in this lower-dimensional space, successfully identifying a non-linear decision boundary that perfectly separates the two classes. The model achieved 100% accuracy on the test set, Figure 11, while using only 33 support vectors—approximately 3% of the training set—compared to larger number required in the previous 12-feature configuration. This reduction in model complexity, combined with perfect classification performance, confirms that the earlier poor results were due to irrelevant features introducing noise. Once reduced to its truly informative dimensions, the SVM becomes an efficient classifier.

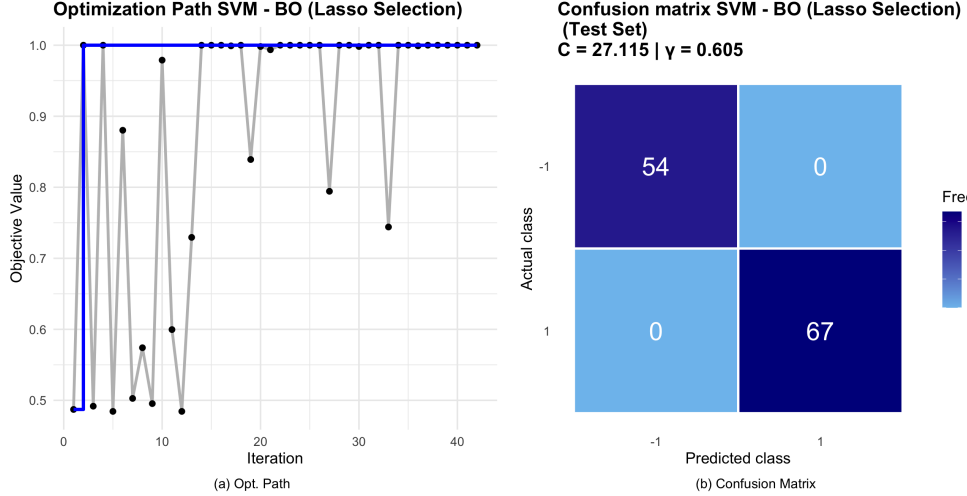


Figure 11: SVM classification results on selected variables

5 Supervised learning techniques: Probabilistic Models

We now move from the crisp predictions of the KNN algorithm and SVM algorithm and propose two different probabilistic prediction models. Given the nature of the models proposed in the following sections, we adopt different strategies regarding feature selection. For Gaussian Process classification, we use only the two selected features (1-2) from the Lasso analysis, as GP methods can be sensitive to high-dimensional noisy feature spaces. For Random Forest, we use all 12 features, as this method is inherently robust to irrelevant variables through its random feature sampling and averaging mechanism.

5.1 Gaussian Process

As first model, we consider Gaussian Process (GP) for binary classification. This process is not only probabilistic but also non-parametric, since it defines a distribution over an infinite-dimensional space of functions rather than a finite set of parameters. This characteristic makes it very powerful in capturing non-linear and complex relationships. Following the variable selection results, we fit the GP using only the two selected features. We employ a Gaussian (RBF) kernel with Automatic Relevance Determination (ARD), which allows the model to learn a separate length-scale parameter for each feature dimension. After training, the optimized length-scale parameters (in log-scale) are shown in Table 1.

Table 1: Log-length-scale parameters for selected features

Feature 1	Feature 2
2.1206	2.1174

These values correspond to actual length-scales of approximately $\ell_1 \approx 8.34$ and $\ell_2 \approx 8.31$. The near-identical length-scales indicate that both features contribute equally to the classification task, which is consistent with our variable selection analysis. The moderate length-scale values suggest that the GP captures smooth, non-linear decision boundaries in the feature space.

The GP successfully captures the complex geometric structure of the data in this two-dimensional space. As shown in Figure 12, the model achieves 100% accuracy on the test set, with all data points correctly classified. This excellent performance demonstrates that the GP provides an effective probabilistic classification solution for this problem.

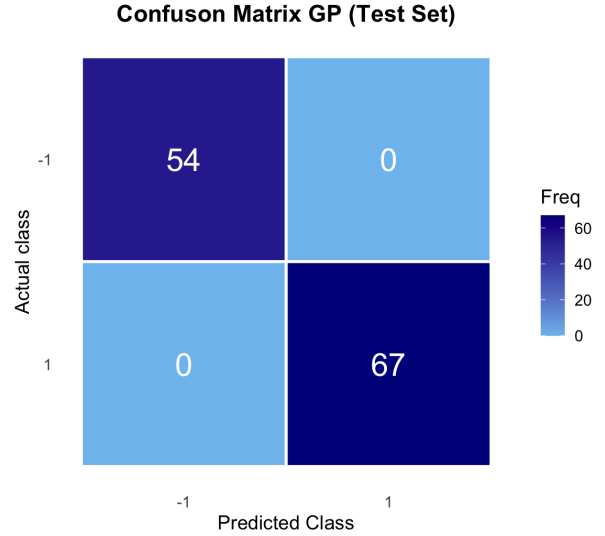


Figure 12: Confusion Matrix GP

5.2 Random Forest

The last model proposed is the Random Forest. In order to explore a substantial part of the large parameter space that underpins a random forest, we paired the model with a Bayesian optimization algorithm. Random forests are powerful yet conceptually easy models. This kind of model can be easily understood by looking at the hyperparameters. A basic RF model requires at least 4 key parameters: number of trees, number of features to be considered, node size, and max nodes. Number of trees tells us how many trees we are going to build, then the number of features tell us how many covariates we need to consider in each tree, node size controls for the depth (how many splits), and max nodes control the maximum number of nodes for each tree. Table 2 shows the values of optimal hyperparameters.

Table 2: Values of the optimal hyperparameters

Ntree	Mtry	Nodesize	Maxnodes
726	13	1	77

Focusing on the results, we observed that the optimal value of node size is 1. This value obtained is also the value which is commonly recommended for binary classification problems, indicates that each tree can grow very deep. This conclusion validates our analysis, given that the value of 1 emerged as the best-performing option among all hyperparameter configurations tested as illustrated in Figure 13(a). Moreover, it can be easily seen that an increase in its value leads to a decrease in the level of accuracy. As said before, with a low value for node size each tree can grow very deep. Despite this depth, the high validation accuracy reached, Figure 13(b), suggests that the Random Forest’s ensemble nature and the max-nodes constraint effectively prevent overfitting, allowing the model to capture complex patterns while maintaining good generalization. Figure 14 shows how this model is able to achieve 100% accuracy.

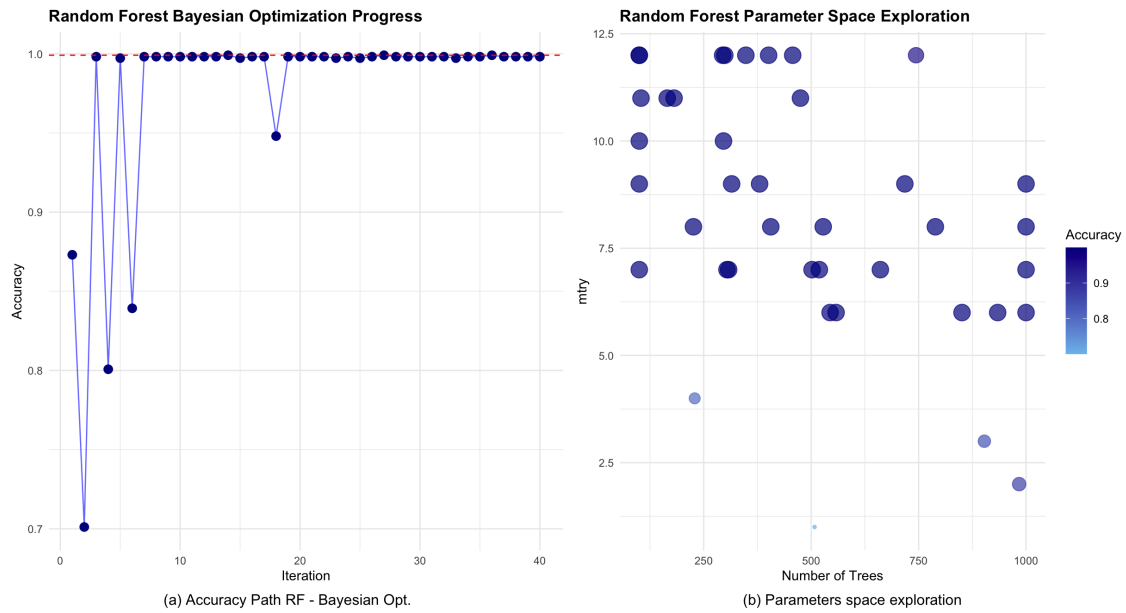


Figure 13: Accuracy and exploration of the parameter space

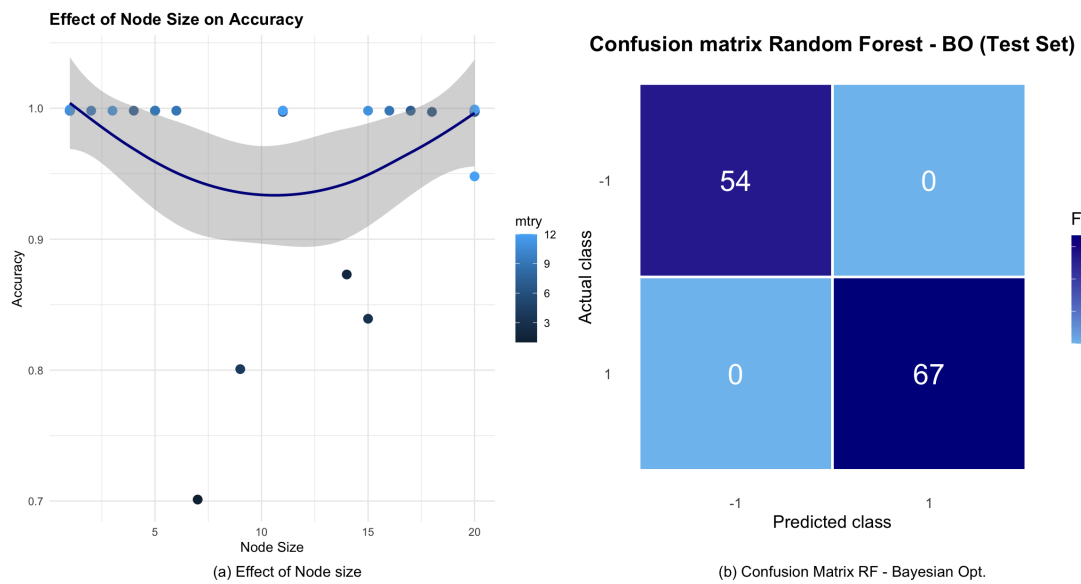


Figure 14: Node size and Confusion Matrix

6 Conclusion

The proposed study implemented four different classification models, along with different optimizations algorithms. The dataset analyzed presents itself with a complex geometry and, as found in the models, non-linear relations. Given those characteristics, simpler models such as KNN and SVM were not able to capture the real relations linking the feature to the class. However, once we reduced the number of features through Lasso regression, the same models are able to capture the relation between the most important features (1 and 2) and the class. Regarding probabilist models, both GP model and RF performed well. The former takes into account the most relevant features, but it has an high computational cost, $O(n^3)$. The latter results to be faster and it considers all the features, since we do not know the Data generation process, not considering all the available features may not be the best approach.

Therefore, RF model is the proposed one. While achieving 100% accuracy would raise strong concerns regarding the risk of overfitting, Random Forest is known for its ability to capture complex relationships and achieve perfect or near-perfect classification. Thanks to the use of Bagging and random feature selection, it successfully mitigates overfitting. It not only excels in accuracy but also successfully balances high performance with computational practicality.

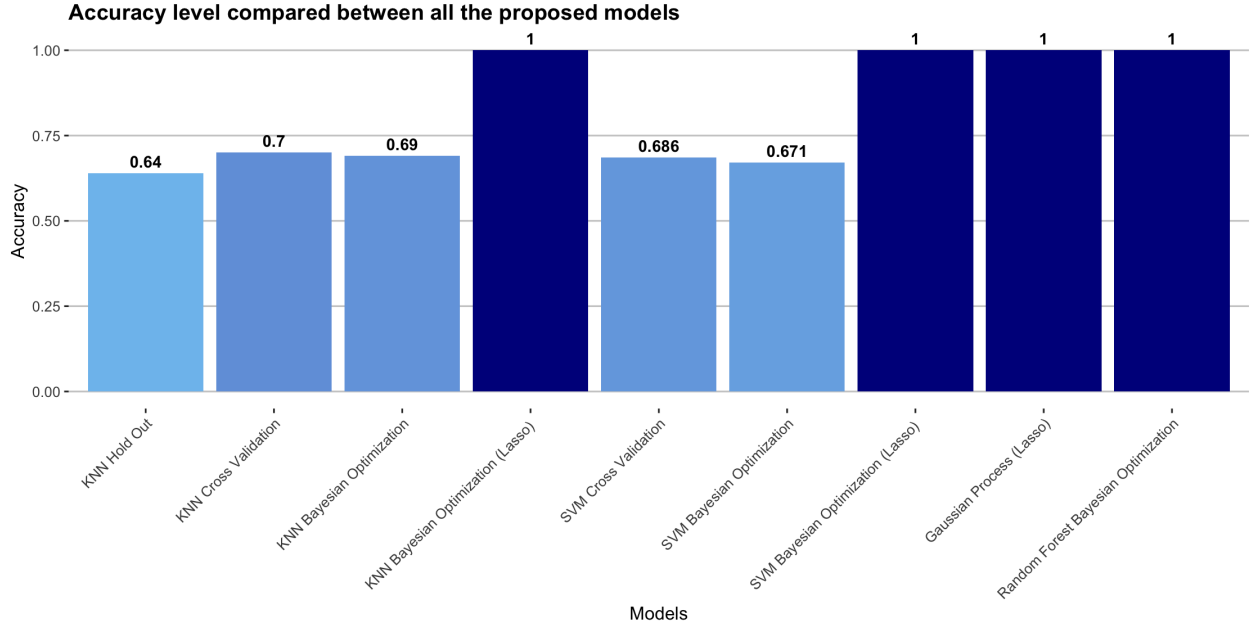


Figure 15: Accuracy comparison